

DRMC Patient Portal Setup and Administration Guide

Create an independent copy of the portal, connect your own Microsoft SQL Server database, establish Admin and staff access, load the approved doctor roster, and register patients against their existing hospital records.

For the client IT team, administrators and developers

Reviewed on **7 October 2026** against GitHub default branch **master**, commit **d86ddb61f05fac0f4bf007932407998d3f790cdb**.

Repository: [DAJabonite drmc patient portal](#)

Commands use PowerShell on Windows unless marked Bash or SQL. Replace example database names, addresses and paths with your own. All screenshots and companion CSV rows are demonstration material.

This guide describes the application at the reviewed commit. A clone contains code and migrations; it does not contain the original developer's database, staff accounts, secrets or encrypted documents. The current application has no live integration with DRMC clinical, pharmacy, billing or social-work systems.

The supplied reference PDF is already substantially complete. Appendix C explains its corrections and the practical additions in this edition.

Contents

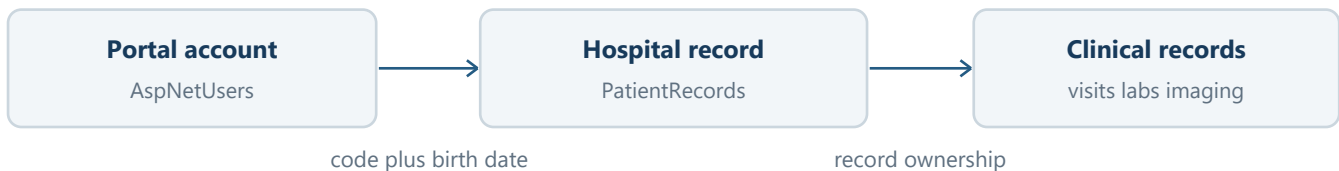
Read chapters 1 through 14 in order for a new installation. Developers should also read chapter 16. Client hosting, recovery and handover are in chapters 17 through 20.

1 Understand the project and setup order	3
2 Install the required tools	4
3 Clone and build your own copy	5
4 Create your own Windows database	6
5 Use Docker or another SQL Server	7
6 Configure secrets and persistent storage	8
7 Start the portal and confirm setup	9
8 Create the first Admin	10
9 Invite staff with the right access	13
10 Prepare the approved doctor roster	15
11 Import patients and clinical data carefully	18
12 Create the hospital patient records	19
13 Issue the patient token and QR	20
14 The patient creates the linked account	22
15 Maintain clinical records and public content	24
16 Developer workflow and test data	25
17 Prepare the client hosting environment	26
18 Back up and recover the installation	30
19 Troubleshoot setup and staff access	31
20 Rehearse the complete workflow and hand over	32
Appendix A Migration and source reference	33
Appendix B Useful addresses	34
Appendix C Review of the supplied reference PDF	35
Appendix D Sources and verification	36

1 Understand the project and setup order

The portal is an ASP.NET Core MVC and Razor Pages application on .NET 10, with ASP.NET Core Identity for accounts and Entity Framework Core 10 for Microsoft SQL Server. It serves public hospital information and a doctor directory, patient account and ID-document management, linked clinical records, and a staff Administration area.

Public pages include hospital information, departments, doctors, advisories, OPD and Malasakit guidance. **Patient pages** show the signed-in account's linked visits, laboratory availability, imaging availability, medications, allergies and activity. **Staff pages** manage the registry, clinical entries, directory content, imports and audit history.



Directory doctors and the code-defined department catalog are separate.

Keep these concepts separate

Concept	What it means
Hospital patient record	A PatientRecords row created or imported by Admin. Clinical entries belong to this row before a patient has a login.
Portal account	An Identity AspNetUsers row. Its email and password are separate from the hospital record.
Hospital record code and QR	A one-use enrollment code assigned to one existing hospital record. Valid code plus matching birth date links a new portal account to that record.
Staff invitation	A separate, email-bound signup code. It grants one selected staff role after email confirmation and authenticator setup.
Department	A fixed catalog in source code. Doctors store its canonical name as text; there is no SQL Departments table.

Complete setup in this order

1. Install tools, clone, restore and build.
2. Select an independent SQL database, configure storage and apply migrations.
3. Run locally and create the first Admin.
4. Invite staff and complete their email confirmation and 2FA.
5. Confirm department names and load the approved doctor roster.
6. Create or import hospital patient records.
7. Verify identity at the desk, issue a code or QR, and complete patient signup.
8. Rehearse the workflow, prepare hosting and complete handover checks.

Result visibility: PatientResults:ShowFullResults defaults to false. The portal can show availability without exposing released lab values or imaging report sections. Keep the default until DRMC approves disclosure.

2 Install the required tools

Tool	Install or check
Git	Install from Git for Windows , or your OS package manager. Check with <code>git --version</code> .
.NET 10 SDK	Install the SDK from Microsoft .NET 10 downloads . A runtime alone does not build the project.
EF command-line tool	This commit uses dotnet-ef 10.0.11. Installation commands are in chapter 3.
SQL Server	Use SQL Server 2022 Express or Developer for local work, or an available client-managed SQL Server.
Database viewer	Use SSMS on Windows, or VS Code with Microsoft's MSSQL extension. SSMS is separate from the SQL engine.
Authenticator	Every Admin and staff member needs a TOTP authenticator, such as Microsoft Authenticator or Google Authenticator.
Docker	Optional alternative to local SQL installation. Use Linux containers for the supplied compose file.

Windows SQL Server Express walkthrough

1. Download the SQL Server 2022 Express installer from Microsoft's [SQL Server downloads](#) or official version-specific download links.
2. Run setup. For a configurable installation, choose the Custom route, then a new stand-alone installation. Select Database Engine Services.
3. Use the named instance **SQLEXPRESS** to follow this guide. An existing instance with another name is usable if you change the connection string.
4. Choose Windows authentication and add your Windows user as a SQL administrator for local development. Finish installation.
5. Install [SQL Server Management Studio](#). Connect to **localhost\SQLEXPRESS**, with Windows authentication.
6. Confirm the Database Engine service is running. SSMS by itself does not supply a SQL Server instance.

These choices follow Microsoft's [SQL Server setup guidance](#).

Current tool choice: Azure Data Studio retired on 28 February 2026. Use SSMS or VS Code with MSSQL for a new setup. See [Microsoft's retirement guidance](#).

SQL Server Developer edition is for development and testing, not a live production database. The client's DBA chooses an edition and capacity suitable for production; see [Microsoft's edition guidance](#).

3 Clone and build your own copy

Open PowerShell in the parent folder where you want your copy. Run:

POWERSHELL

```
git clone https://github.com/DAJabonite/drmc-patient-portal.git
cd drmc-patient-portal
git branch --show-current
git log -1 --oneline
git remote -v
dotnet --version
```

A normal clone starts on **master**. This guide was checked at **d86ddb6**. If GitHub has newer commits, read the updated README and migration history before applying these instructions. To reproduce this edition's exact code on a separate development branch:

POWERSHELL

```
git checkout -b setup-guide-review d86ddb6
```

Restore packages and build

POWERSHELL

```
dotnet restore DrmcPatientPortal.slnx
dotnet build DrmcPatientPortal.slnx -c Release -warnaserror
dotnet tool install --global dotnet-ef --version 10.0.11
dotnet ef --version
```

If dotnet-ef is already installed at 10.0.11, skip installation. If another version is installed, use **dotnet tool update --global dotnet-ef --version 10.0.11**. Reopen the terminal if a newly installed tool is not found. A successful build reports zero errors; this command treats warnings as errors.

There is no separate npm build step for this version. The repository contains its browser assets. The application project is **src/DrmcPatientPortal** and the solution file is **DrmcPatientPortal.slnx**.

Optional independent GitHub repository

Cloning is enough to run an independent local installation. If the client also needs a separate code repository, create a fork or an empty repository in the client's GitHub account. For an empty repository:

POWERSHELL

```
git remote rename origin upstream
git remote add origin <client-repository-url>
git push -u origin master
```

Replace the placeholder first. Use the actual current branch if it is not master. Future updates can be fetched with **git fetch upstream**, reviewed, and merged through the team's normal process. Database contents and server settings are not transferred by Git.

4 Create your own Windows database

Each developer should use a separate database. These examples use **DrmcPatientPortal_Dev_YourName** on **localhost\SQLEXPRESS**. Replace YourName consistently. This avoids sharing the repository's default **DrmcPatientPortal_Dev** unintentionally.

Set the connection explicitly

Run from the repository root in the same terminal used for migrations and the app:

POWERSHELL

```
$project = 'src/DrmcPatientPortal'
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:AdminBootstrap__Enabled = 'false'
$env:DevelopmentFixtures__Enabled = 'false'
$env:ConnectionStrings__DefaultConnection = (
    'Server=localhost\SQLEXPRESS;' +
    'Database=DrmcPatientPortal_Dev_YourName;' +
    'Integrated Security=True;Encrypt=True;' +
    'TrustServerCertificate=True'
)
$target = [System.Data.SqlClient.SqlConnectionStringBuilder]::new(
    $env:ConnectionStrings__DefaultConnection
)
Write-Host "Server: $($target.DataSource)"
Write-Host "Database: $($target.InitialCatalog)"
if ($target.DataSource -ne 'localhost\SQLEXPRESS' -or
    $target.InitialCatalog -ne 'DrmcPatientPortal_Dev_YourName') {
    throw 'Unexpected migration target'
}
dotnet ef database update --project $project -- --environment Development
```

The check prints server and database, never a password. Adjust both expected values if using another target. In a PowerShell host without System.Data.SqlClient, verify the connection's target in SSMS before running the migration command.

EF can create the database when your login has permission. It creates tables from all eight committed migrations. Migrations are an explicit setup step; starting the app does not apply them automatically.

If a DBA creates the empty database first

It must use **Latin1_General_100_BIN2** collation. The initial migration rejects another database collation. Run in SSMS as a suitably privileged user:

SQL

```
CREATE DATABASE [DrmcPatientPortal_Dev_YourName]
COLLATE Latin1_General_100_BIN2;
```

Then run the EF update against it. Do not apply these migrations to a hospital system database with unrelated tables. This portal owns a separate schema and is not an automatic migration from older SQLite data.

Expected result: SSMS shows the new database, Identity tables, Doctors, PatientRecords, PatientRegistrationCodes, StaffInvitations and __EFMigrationsHistory. Verify migration IDs with Appendix A's query before creating accounts.

5 Use Docker or another SQL Server

The supplied **compose.yaml** starts SQL Server 2022 Developer, publishes it on **127.0.0.1:1433**, and persists data in a Docker volume. It starts the database service only; run the .NET application separately.

Windows PowerShell with Docker

POWERSHELL

```
$env:MSSQL_SA_PASSWORD = Read-Host 'Local SQL container password'
docker compose up -d --wait
docker compose ps
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:ConnectionStrings__DefaultConnection = (
    'Server=localhost,1433;Database=DrmcPatientPortal_Dev_YourName;' +
    'User Id=sa;Password=' + $env:MSSQL_SA_PASSWORD +
    ';Encrypt=True;TrustServerCertificate=True'
)
dotnet ef database update --project src/DrmcPatientPortal -- --environment Development
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

Use a unique local password meeting SQL Server complexity rules. Read-Host here accepts plain text; type it only in a private local terminal. The example uses sa only for disposable development. Live installations use a dedicated runtime login.

Linux or macOS terminal

BASH

```
read -rsp "Local SQL container password: " MSSQL_SA_PASSWORD
printf '\n'
export MSSQL_SA_PASSWORD
docker compose up -d --wait
export ASPNETCORE_ENVIRONMENT=Development
export ConnectionStrings__DefaultConnection="Server=localhost,1433;\
Database=DrmcPatientPortal_Dev_YourName;User Id=sa;\
Password=$MSSQL_SA_PASSWORD;Encrypt=True;TrustServerCertificate=True"
dotnet ef database update --project src/DrmcPatientPortal \
    -- --environment Development
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

The Bash connection uses intentional backslash continuations inside the quoted string. Do not add spaces after those backslashes.

Practical limits

- Stop containers with **docker compose stop**; resume with **docker compose start**. **docker compose down** removes the container but preserves the volume.
- **docker compose down -v deletes the database volume**. Use it only to discard a confirmed throwaway environment.
- If port 1433 is occupied, use your installed SQL Server or choose a different host port and update the connection.
- The image is linux/amd64. Microsoft supports SQL Server containers on Linux x86-64 hosts; ARM emulation is not a supported deployment. Apple silicon and Windows ARM developers can use a reachable x86-64 SQL server. See [Microsoft's container guidance](#).

Manual ID entry works if native Tesseract OCR is unavailable. Validate native OCR dependencies before relying on photo extraction in a deployment.

6 Configure secrets and persistent storage

Configuration is loaded from `appsettings.json`, the environment-specific file, Development user-secrets and environment variables; command-line overrides can take precedence. Environment variables use **double underscores** between nested keys.

The tracked production connection is a placeholder. The Development connection targets `localhost\SQLEXPRESS` and `DrmcPatientPortal_Dev`. Set your own connection explicitly so a different shell, user-secret or launch method does not select the wrong database.

Persist local settings when needed

POWERSHELL

```
$project = 'src/DrmcPatientPortal'
$localConnection = '<your-local-connection-string>'
dotnet user-secrets set "ConnectionStrings:DefaultConnection" $localConnection --project $project
```

The project already has a `UserSecretsId`. Copies under the same OS user share that identifier and those secrets. For independent clones, use per-terminal environment settings, or deliberately assign each project a distinct secrets identifier.

User-secrets are a Development convenience, not encrypted production secret storage. Do not commit passwords, invitation slips, patient data, real roster files or key files. Remove a saved local setting with **`dotnet user-secrets remove "Setting:Name" --project src/DrmcPatientPortal`**.

Choose stable folders

For local work, defaults use ignored `App_Data` folders under the application. On a server, use absolute folders outside the deployment and web roots:

POWERSHELL

```
$env:DataProtection__KeyRingPath = 'C:\DRMCDData\Keys'
$env:PatientDocuments__RootPath = 'C:\DRMCDData\PatientDocuments'
$env:PatientDocuments__TemporaryPath = 'C:\DRMCDData\TempUploads'
$env:Imports__StagingPath = 'C:\DRMCDData\ImportStaging'
```

Grant the application identity Modify access to these folders. Independent installations need different storage roots. Replacing a published application folder must not erase storage or keys.

Setting	Meaning and default
<code>DataProtection:ProtectKeysWithDpapi</code>	false. Optional Windows current-user protection; read chapter 18 before changing it.
<code>PatientDocuments:MaximumFileSizeMb</code>	10 MB per upload.
<code>PatientDocuments:TemporaryFileLifetimeMinutes</code>	30-minute staged document lifetime.
<code>Imports:StagingPath</code>	If unset, current user's local application-data <code>DRMC/ImportStaging</code> directory. Set explicitly on servers.
<code>Identity:RequireConfirmedAccount</code>	true. Keep email confirmation enabled.
<code>PatientRegistration:RequireHospitalRecordCode</code>	true. Enrollment requires an existing hospital record.
<code>PatientRegistration:CodeLifetimeDays</code>	7 days, bounded to 1 through 30.
<code>StaffInvitations:LifetimeDays</code>	3 days, bounded to 1 through 14 for accepting an unused invitation.

7 Start the portal and confirm setup

With the database migrated and environment variables still set:

POWERSHELL

```
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

Open **http://localhost:5095**. Stop with **Ctrl+C**. On Windows, **.\run.ps1** is another launcher, but it does not create a database or apply migrations.

For local HTTPS:

POWERSHELL

```
dotnet dev-certs https --trust  
dotnet run --project src/DrmcPatientPortal --launch-profile https
```

Open **https://localhost:7104**. This developer certificate is for your development machine, not production.

Check the first run

1. The public home page, doctor directory and guidance pages load.
2. **Create account** and **Log in** are visible. A new database has no doctors, advisories or clinical rows until you load them; empty pages can be expected.
3. Patient signup requires a hospital record code by default. Admin uses the separate bootstrap path in chapter 8.
4. Watch the console. Development confirmation and invitation emails are logged under **[EMAIL OUTBOX]**, not delivered through SMTP.
5. In SSMS, verify all eight migrations and the target database. Create content through the app rather than inserting Identity users or roles manually.

The app creates its four staff role definitions at startup, but provides no default Admin password. A password alone never grants staff access.

Local QR and phone testing

localhost on a phone means the phone itself, so a QR generated while staff browse localhost cannot reach your development PC. A private/incognito window on the same PC can test record linking. For a phone rehearsal use a trusted, reachable test HTTPS hostname with appropriate network access and certificate setup; generate the slip through that address.

In production, staff must use the real public HTTPS address to generate invitations and slips. Chapter 17 explains scheme and host checks.

Environment matters: Development selects console notifications and permits explicitly enabled fixtures. Production or Staging also requires SMTP and HTTPS SMS configuration and secure hosting.

8 Create the first Admin

The first Admin cannot receive an invitation from an Admin who does not yet exist. Enable the temporary bootstrap for one intended email. Until any Admin exists, only that email may sign up without a hospital record code.

Start bootstrap and create the account

POWERSHELL

```
$env:AdminBootstrap__Enabled = 'true'
$env:AdminBootstrap__Email = Read-Host 'First Admin email'
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

Use the same database and storage settings as chapters 4 and 6. Stop an existing app process before setting these values. Startup waits for the configured account; it does not create a user or password.

1. Open **Create account** at **/Identity/Account/Register**.
2. Read the privacy notice and terms, scroll to the end, check consent and select **Agree and continue**.
3. Complete **ID selection** → **ID photo** → **Review & edit** → **Credentials**. The page still calls this a patient account even when used for Admin or staff.
4. Supply ID type and number, first and last name, complete address and mobile. **Enter details manually** skips photo upload; photos are optional. Birth date is optional for bootstrap.
5. Use the bootstrap email. Leave **Hospital record code** empty. Enter and confirm a strong password, then select **Create account**.

Confirm email

In Development, find **[EMAIL OUTBOX]** in the console. Copy the URL inside the email's href attribute and replace HTML **&** separators with **&**. Open the corrected URL.

With SMTP configured, use the confirmation link in the delivered email. The success page says **Your email is confirmed**; select **Continue to sign in**. Confirmation does not automatically sign you in.

Enable the authenticator before restarting

1. Sign in and open **Two-Factor Auth (2FA)** under account management, or **/Identity/Account/Manage/TwoFactorAuthentication**.
2. Select **Set up Authenticator App (Google Authenticator / Microsoft Authenticator)**.
3. Scan its authenticator QR or enter the manual setup key in the authenticator.
4. Enter the six-digit code and select **Verify & Activate 2FA**.

Do this in the same running session. If bootstrap restarts after the account exists but before email confirmation and 2FA are complete, startup stops. Recovery is explained on the next page.

8 Finish Admin setup and verify access

Once email is confirmed and 2FA enabled, stop the app. Restart with the same bootstrap variables. Startup now grants that existing account the **Admin** role.

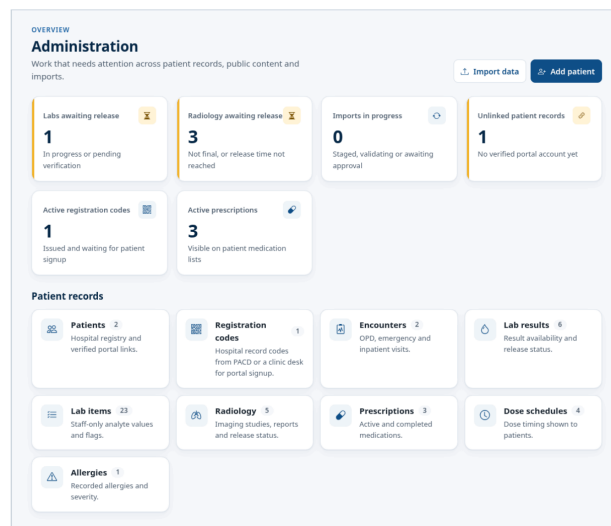
Remove bootstrap and sign in again

POWERSHELL

```
# Stop the app with Ctrl+C before these commands.
Remove-Item Env:AdminBootstrap__Enabled -ErrorAction SilentlyContinue
Remove-Item Env:AdminBootstrap__Email -ErrorAction SilentlyContinue
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

Remove bootstrap from user-secrets or server settings too if stored there. Clearing terminal variables does not remove a saved value elsewhere.

Sign out, then sign in using password **and an authenticator code**. Leave **Remember this device for 30 days** unchecked. Open **/Admin** or **Administration**.



Administration dashboard after an eligible Admin sign-in. Demonstration screenshot from the supplied reference PDF.

If Admin access is denied

Staff requests check current role, confirmed email, enabled 2FA, no lockout and an MFA sign-in claim. Password-only and remembered-device sessions do not qualify. Choose **Forget this browser for 2FA**, sign out, and sign in with a fresh authenticator code.

If bootstrap blocks startup because the account is not ready, disable it, restart, finish email confirmation and 2FA, then enable bootstrap and restart once to promote.

Additional Admins and recovery

Admin is unavailable through invitations or Staff access grants. Create the second person's account through a staff invitation or verified patient signup, confirm email and enable 2FA. An operator runs bootstrap with that existing email, restarts to promote, removes bootstrap, and the person signs out and in with MFA.

The first-signup exemption closes once an Admin exists; startup promotion of an existing eligible account still works. Restrict bootstrap settings to deployment operators.

Enrollment does **not automatically display recovery codes** in this custom UI. Generate them deliberately through the built-in page and rehearse recovery as explained next.

8 Generate and use recovery codes

The account management screen does not link the recovery-code controls, but the built-in Identity pages work at this commit. This edition verified generation, recovery sign-in, Admin access and one-use consumption against the installed application.

Generate the codes while signed in

1. Enable 2FA first, then open **/Identity/Account/Manage/GenerateRecoveryCodes** on your own portal.
2. Select **Generate Recovery Codes**.
3. The next page shows **ten** codes. Store them privately in the organization's approved recovery method, separately from the authenticator device.

These are account secrets. Generating a new set replaces the old set. Custom authenticator enrollment alone does not generate them. Staff and patients with 2FA can use this built-in account workflow.

Recover a sign-in without the authenticator

1. Open Log in and enter the account's email and password.
2. When the login reaches the pending authenticator stage, open this path in the **same browser/session**:

TEXT

```
/Identity/Account/LoginWithRecoveryCode?returnUrl=%2FAdmin
```

3. Enter one unused code in **Recovery Code** and submit.
4. The recovery sign-in supplies MFA proof and can enter Admin if the account still satisfies current role, email, 2FA and lockout requirements.
5. Mark that code used; it cannot be used again. Restore or reconfigure the authenticator through account management under the approved identity-verification procedure.

For a patient recovery sign-in, use **returnUrl=%2FPatient%2FHome** instead. A recovery code does not replace the initial password step or bypass role restrictions.

Rehearse before handover

Use a synthetic staff account to generate ten codes, sign out, sign in using password plus one recovery code, open its permitted staff area, and verify nine codes remain. Confirm that replay fails. A live rehearsal consumes a real code; update the stored inventory.

If no working authenticator, signed-in account or recovery codes remain, the current UI provides no general operator reset of another person's MFA. Escalate to the responsible deployment/identity operator after identity verification; do not disable staff eligibility checks.

This behavior is defined by the built-in **Identity recovery-code generation** and **Identity sign-in implementation**, and was exercised separately from the existing 111-test suite.

9 Invite staff with the right access

An Admin assigns staff access; public patient signup does not offer staff roles. Most staff have no hospital patient record, so use **Staff invitations**, not a fabricated hospital record code.

Role	Sections available
Admin	All staff sections, including registry, doctors, advisories, clinical data, imports, audit, staff access and invitations.
LabStaff	Dashboard, LabResults and LabResultItems.
RadiologyStaff	Dashboard and RadiologyStudies.
PatientServicesStaff	Dashboard and RegistrationCodes.
Ordinary patient	No Administration access.

All staff need confirmed email, enabled 2FA, a fresh MFA sign-in, current role membership and no lockout. The sidebar shows permitted sections. Patient services staff can issue codes for existing records but cannot create records or correct their birth dates.

The Admin sends an invitation

1. Sign in as Admin with an authenticator code.
2. Open **Staff invitations** → **Invite staff**, or **Staff access** → **Invite staff**.
3. Enter **Work email address** and select **Laboratory staff**, **Radiology staff** or **Patient services staff**.
4. Select **Send invitation**. Admin is never granted by invitation.
5. The created invitation page shows the one-time code, QR, validity and **Print**. Print or show it now; the full code cannot be retrieved later.

STAFF INVITATIONS

Invite staff

The staff member gets a one-time signup link by email, and you can also print or show it. It works only with this email address and expires in 3 days.

Invitation

Work email address

jose.lim@demo.test

The staff member must sign up with exactly this address.

Staff role

Laboratory staff

Granted automatically after the account confirms its email and turns on two-factor authentication. Admin access is never granted by invitation.

Invite staff screen with work email and selected role. Demonstration screenshot from the reference PDF.

The signup link is emailed. In Development, retrieve it from the console. A mail delivery failure does not invalidate the invitation; use the printed code or QR, or revoke and issue a replacement. If that email already has an account, use Staff access instead.

Unused invitations normally expire after **3 days**. The code is bound to the normalized email address, with case-insensitive matching. The form accepts valid email syntax; a hospital-domain address is an organizational choice, not an enforced domain restriction.

9 Complete staff signup and manage access

The invited staff member completes setup

1. Open the invitation link or QR. It leads to **/Identity/Account/Register#invite=....**
2. Accept privacy and terms and complete the same ID/profile wizard described in chapter 8. An ID type, ID number, name, address and mobile are still required. Photo and birth date are optional for invitation signup.
3. On Credentials, use the invited email and **Staff invitation code**. If entering a code manually, select **DRMC staff with an invitation? Use it instead**.
4. Create the account and confirm email.
5. Sign in and enable the authenticator. Email confirmation and 2FA together trigger the selected role's automatic grant, whichever is completed last.
6. Sign out and sign in with password and authenticator code. Open Administration.

Interpret invitation states

State	Meaning and action
Invited	Not accepted. Revoke if sent to the wrong person; reissue if lost or expired.
Awaiting setup	Signup accepted, but email confirmation or 2FA remains. Finish setup; do not create another account.
Activated	Role granted. Future access changes use Staff access.
Revoked	Cannot accept or activate. Review before replacing.
Expired	An unused invitation passed its acceptance deadline. Issue a new one.

The expiry applies to acceptance. An accepted invitation can remain Awaiting setup beyond its original validity date and still activate unless revoked. Admin can revoke an invitation while Invited or Awaiting setup; a revoked one never grants its role.

Staff who already have an account

In **Staff access**, change **Staff accounts** to **All accounts**, search by email or name, and confirm email and 2FA requirements are met. Use **Grant laboratory staff**, **Grant radiology staff** or **Grant patient services staff**. Multiple staff roles can be assigned when responsibilities require them.

Use the corresponding **Revoke** action when access ends. Grants and revocations refresh the security stamp, recheck authorization and record the change in staff audit history. When a person leaves, also revoke pending invitations and review their assigned roles.

Passwords require 8 through 100 characters, uppercase, lowercase, a digit and a symbol. Five failed attempts can lock an account for 15 minutes. The staff MFA flow uses an authenticator; SMS OTP login is not implemented.

10 Prepare the approved doctor roster

The client supplied names and departments. **The application also requires a Title**, such as the person's hospital-approved credentials. Obtain it from the client; do not invent MD, fellowships or specialist credentials.

This review did not find an authoritative client roster in the repository or the supplied attachment. Example names in DbInitializer and department chair descriptions are demonstration content. Use the client's approved source list to prepare the import.

Confirm these fields with the client

Field	Required	What to supply
FullName	Yes	Approved public display name.
Title	Yes	Approved title or credentials.
Department	Yes	Exact canonical department below.
SubSpecialty	No	Confirmed area of practice.
ClinicRoom	No	Approved room/location.
ScheduleSummary	No	Approved free-text clinic schedule.
OffersTeleconsult	No	Explicit true or false; do not assume availability.
Biography	No	Approved public biography.
PrcLicenseMasked	No	Approved masked license display only.
IsActive	Set explicitly	true publishes in listings; false excludes from listings.

Canonical department names at this commit

Use this exact name	Use this exact name
Internal Medicine	Surgery
Pediatrics	OB-Gyne
Family & Community Medicine	Anesthesiology
Ophthalmology	Radiology

Department case, spacing and punctuation must match exactly in doctor forms and imports. For example, **OB-GYN** is not **OB-Gyne**. Agree on a mapping with the client rather than changing the source roster silently.

Review repeated name/department pairs before upload. Imports reject duplicates against existing rows and within the batch, ignoring case after trimming the full name. Manual forms do not implement the same duplicate guard, so review existing doctors before adding.

Publishing caution: Active controls directory and department listings. At this commit, the individual /Directory/Doctor/{id} route does not filter inactive doctors. Deactivation should not be treated as making a direct profile URL private.

10 Update departments and add doctors manually

Departments are defined in `src/DrmcPatientPortal/Models/ClinicalDepartment.cs`, in `ClinicalDepartments.All`. There is no Departments SQL table, department import or Admin department editor.

If the client's departments differ

- Map alternative spellings to approved existing canonical names when they mean the same department.
- If a new department is required, a developer adds it to the catalog, rebuilds, reviews affected pages and redeploys.
- Department metadata includes description, icon, head, location, extension and services. Ask DRMC to confirm these values before publication.
- Renaming a catalog item does not rename text already stored in Doctors, ClinicalEncounters or Prescriptions. Prepare a controlled data change for all affected rows and test filters and editing afterward.

Example C# catalog entry for a new department, with placeholders to replace:

CSHARP

```
new(  
    "Dermatology",  
    "Replace with the approved service description.",  
    "stethoscope",  
    "Replace with the approved department head",  
    "Replace with the approved location",  
    "Replace with the approved extension",  
    new[] { "Replace with an approved service" }  
),
```

Adding a static catalog item alone does not require a new database table or schema migration. Keep the department named exactly **Radiology**: the patient imaging pages retrieve its metadata through a catalog lookup that requires that name.

Add a small number of doctors through Admin

1. Open **Public content** → **Doctors** → **Add doctor**.
2. Enter Full name, Title and select Department.
3. Fill only confirmed optional details. Choose **Offers teleconsultation** and **Active** deliberately.
4. Select **Save doctor**.
5. Open **/Directory** without signing in. Search by name, filter by department and verify the public display.

Use **Edit** for corrections to an existing doctor. An old edit form can fail its version check if someone changed the row; reload and review rather than overwriting their changes.

Directory doctor rows are not login accounts. Adding a doctor does not grant staff access. Clinical records preserve physician-name snapshots, so editing a public directory name does not rewrite historical visit, laboratory or prescription names.

10 Load doctors from CSV

For the supplied roster, use the application's exact **Doctors CSV** template.

1. Open **Operations** → **Imports** and download **Doctors CSV**.
2. Keep the ten headers and their order exactly. Fill approved roster rows.
3. Save a comma-separated **UTF-8 CSV**. Quote values containing commas or line breaks; double any quote characters inside a quoted value.
4. Select **Doctors_v1 (.csv)** as Template, choose the file and select **Upload**.
5. Reconciliation has no patient source groups for a doctors-only file. Select **Save mappings and dry run**.
6. Wait for the dry run, read every error and compare proposed counts with the roster.
7. At Ready for approval, check **Confirm the reconciled patient records and proposed counts**, then select **Approve batch**.
8. Wait for **Succeeded**, confirm counts in Admin and verify the public directory.

Exact header order

Position	Header	Position	Header
1	FullName	6	ScheduleSummary
2	Title	7	OffersTeleconsult
3	Department	8	Biography
4	SubSpecialty	9	PrcLicenseMasked
5	ClinicRoom	10	IsActive

Always set IsActive explicitly. Empty Boolean cells become false, including IsActive. Lowercase **true** and **false** are the documented values; the Boolean parser also accepts their case variants.

The companion **setup-guide-samples/doctors-demo.csv** has the exact one-line header and two fictional rows. It is for a throwaway rehearsal, not the client's live directory. This PDF lists columns separately so a wrapped printed CSV header cannot be mistaken for valid file contents.

Fixing an import

The importer only creates records. It does not update, merge or skip duplicates. To correct the approved source file, cancel the old batch where permitted, fix the file and upload again. Do not approve a second copy of an already successful import.

A failed batch writes no partial business rows. If the approved roster changes after a successful load, edit the existing rows through Admin or prepare a separately reviewed data-maintenance process; importing them again is not an update.

11 Import patients and clinical data carefully

Workbook v1, downloaded from Imports, contains these nine named sheets. Use **Workbook_v1** for an .xlsx upload. Preserve sheet names, exact header case and column order.

Sheet	Records
Patients_v1	New hospital registry records
Doctors_v1	Public directory doctors
PublicAdvisories_v1	Public notices
ClinicalEncounters_v1	Visits
LabResults_v1	Lab headers and availability
LabResultItems_v1	Parameters belonging to a lab result
Prescriptions_v1	Prescriptions
MedicationDoseSchedules_v1	Dose times belonging to a prescription
PatientAllergies_v1	Allergies

There is **no RadiologyStudies_v1** import at this commit. Enter dedicated imaging studies through **/Admin/RadiologyStudies**. A lab category named RadiologyImaging does not create those studies.

Reconcile ownership before approval

Every clinical source group uses **SourcePatientKey**. Source names and birth dates help the reviewer; they do not automatically match a patient. Explicitly choose an existing hospital record or confirm creation of a new one.

Patients_v1 rows create new registry records and require **Confirm a new hospital record** for each group; they cannot update an existing patient. For clinical rows mapped to an existing patient, use the appropriate clinical template rather than adding a Patients row to overwrite it.

Save mappings on each reconciliation page, rerun the dry run and resolve all groups. Clinical child rows also need exact source parent references. Parent reference forms are **batch:<SourceRecordKey>** and **existing:<hospital reference>**. The batch and existing prefixes are lowercase. Use the downloaded templates and [repository import contract](#) for the full clinical column lists.

Formats and boundaries

- Upload limit: **20 MB** and **10,000 data rows** across the batch.
- Workbooks must contain values only. Formulas, macros, external links, encrypted or legacy OLE workbooks are rejected.
- Store hospital numbers and source identifiers as **text cells** to preserve leading zeros.
- Dates use **yyyy-MM-dd**; clinical date/time uses **yyyy-MM-ddTHH:mm:ss**, without a timezone suffix.
- Clinical timestamps are Manila wall time. Publication, audit, import and system timestamps are UTC.
- Unknown columns, Identity users, portal ownership and arbitrary column mappings are rejected.

Imports do not link portal accounts or issue patient codes. Approval and execution run through the background worker, recheck staff eligibility and commit the whole batch with its record audit entries.

12 Create the hospital patient records

A hospital record must exist in **this portal's database** before staff can issue its signup code. Cloning the app does not copy the hospital's patient population or connect a hospital information system automatically.

Add one registry record

1. Sign in as Admin and open **Patient records** → **Patients** → **Add record**, or use **Add patient** on the dashboard.
2. Enter the verified **Full name**.
3. Enter the verified **Birth date**. It is optional on a record but required before issuing a registration code, must not be in the future, and must match the patient's signup entry.
4. Enter **Hospital number** if supplied by the hospital. Keep leading zeros. It is optional, unique ignoring case when present, and is never generated by the portal.
5. Save and confirm the record's details and unlinked status.

Only Admin can create or edit registry records. Patient services staff should ask Admin to correct missing or incorrect registry information.

Create several records by import

Download **Patients CSV** or use Patients_v1 in the workbook. The header order is:

Position	Column	Purpose
1	SourcePatientKey	Required source grouping identifier
2	SourcePatientName	Optional review hint
3	SourceBirthDate	Optional review hint
4	FullName	Required new registry name
5	DateOfBirth	Authoritative registry birth date
6	HospitalNumber	Optional existing hospital-issued number

Confirm a new record for each source group, dry-run and approve. If records already exist, reconcile clinical files to those existing records instead of creating duplicate registry entries.

How the data belongs together

PatientRecords.Id identifies the hospital record. Visits, labs, imaging, prescriptions and allergies use **PatientRecordId**. Lab parameters belong to lab results; dose schedules belong to prescriptions.

PatientRecords.PortalUserId is the optional link to one Identity account. Code redemption fills it with the new account's user ID. Patient queries use this ownership link, not a name search.

Do not create matching by display name, similar email or hospital number alone. Imports do not populate this account link. Staff linking is a separate verified action.

If a patient already has an account, use Admin's **Verified portal link** with an unambiguous, email-confirmed account and a checked identity-verification confirmation. A registration code is for creating a new account, not adding a code to an existing login.

13 Issue the patient token and QR

The application's **Hospital record code** is the patient enrollment token. Its QR is a convenient way to enter that same code; the printed code and QR are alternatives, not two required credentials.

At PACD or the clinic desk

1. Sign in as Admin or Patient services staff with an authenticator code.
2. Open **Registration codes** → **Issue code**, or the **Registration code** action on a patient's details page.
3. Search by patient name or hospital number and select the correct hospital record.
4. Compare the person's valid ID with the record, especially its birth date. Resolve discrepancies through Admin before proceeding.
5. Choose the **Issuing point** and check **Staff checked the patient's valid ID against this record**.
6. Select **Issue code**, then **Print slip** or show the code and QR immediately.

Supported issuing points are **PACD (Public Assistance and Complaints Desk)**, **Red Star Clinic**, **OPD clinic**, **Emergency department**, **Admitting / Inpatient services**, and **Medical Social Service (Malasakit Center)**.

Issuance rules

Rule	Operational meaning
Existing record	Code refers to exactly the selected PatientRecords row.
Birth date present	Add or correct it through Admin first.
Record unlinked	An already linked record cannot receive a code for a second account.
One active code	Reissuing revokes any earlier active code for that record.
Default validity	7 days, configurable within 1 through 30. Expiry is shown in Philippine time.
One-time display	The full code is shown only in the issued response. It cannot be recovered from the list later.
Audit	Issue and revoke actions are recorded in staff audit history.

The database stores the code's **SHA-256 hash**, last five characters, patient record ID, issuer, issuing point and status times. It does not store the plaintext code. This protects stored token values; staff must still keep the issued slip private.

If a slip is lost or handed to the wrong person, revoke its active code and issue a replacement after checking identity again. The five-character hint in the list helps find the code; it cannot reconstruct the original.

13 Read the slip and understand the QR

REGISTRATION CODES

Code issued

Print or show this slip now. For security, the full code is shown only once and cannot be displayed again.

← Done Print slip

The code was issued and the action was recorded in the audit history.

DAVAO REGIONAL MEDICAL CENTER · PATIENT PORTAL

Your hospital record code

PEVA5 - NF536 - DX5G3

PATIENT	ISSUING POINT
Juan Dela Cruz	Red Star Clinic

VALID UNTIL

October 14, 2026 12:10 PM (Philippine time)

1. Scan the QR code with your phone camera, or go to the portal and choose **Create account**.
2. On the **Credentials** step, enter this code in **Hospital record code**.
3. Enter the same date of birth that is on your hospital record. The code works once.

Keep this slip private. DRMC staff will never ask you for this code by phone, text or social media.

Example issued hospital record code and QR slip. Demonstration only; the pictured code is not a code for a live patient.

The QR opens the registration page at the portal address used by the issuing staff member, with a fragment similar to:

TEXT

```
https://<your-portal-host>/Identity/Account/Register#code=<one-use-code>
```

The portion after # is a browser URL fragment. Browsers do not send it as part of the page-request URL. Registration JavaScript reads the code, fills the Hospital record code field and removes the fragment from the visible address/history.

On final signup, the form sends the code to the server over the normal form POST so it can be verified. The QR is not an offline identity check and does not contain the patient's clinical records.

What the code validates

1. The server normalizes and hashes the entered code.
2. It locates the code row and its specific patient record.
3. The code must be unused, unrevoked and unexpired; the patient record must remain unlinked.
4. The signup birth date must equal the birth date stored on that record.
5. Successful redemption marks the code used and links that record to the new portal user.

Names are not compared to select a record. Staff identity verification and the birth-date check support correct enrollment; the code row supplies the record ID.

Common typing questions

The display format is **XXXXXX-XXXXX-XXXXX**. Input ignores case, spaces and hyphens; O is interpreted as 0 and I or L as 1. A malformed format receives format feedback. A valid-format code can still fail if it is expired, revoked, used or associated with another birth date.

Host check: Scan a freshly generated slip. It must open the real patient-reachable HTTPS host, not localhost or a staff-only internal name. Generate a replacement through the correct portal address if the earlier link cannot reach the app.

14 The patient creates the linked account

1. Give the verified patient their private code slip. They can scan it with their phone camera or visit the portal and select **Create account**.
2. Accept privacy and terms, then complete **ID selection** → **ID photo** → **Review & edit** → **Credentials**. Supply the ID type and number, first/last name and complete address. Photo upload is optional; **Enter details manually** is available.
3. On Review & edit, enter the **same birth date as the hospital record**. The name and address are account profile fields; they are not automatic record-matching keys.
4. On Credentials, confirm the **Hospital record code** was prefilled, or type it manually.
5. Enter email, Philippine mobile number, password and matching password confirmation. The mobile expects **10 digits starting with 9** after the displayed +63 prefix. Passwords require **8–100 characters**, with uppercase, lowercase, a digit and a symbol.
6. Select **Create account**, confirm the email, sign in and open Patient home.

Progress bar: ID selection (✓), Photo skipped (○), Review & edit (✓), **Credentials (4)**

Your hospital record code was added from the QR code. Complete the steps to create your account.

Set up your login credentials

Provide your email address, Philippine mobile number, and a strong password.

Email address

Mobile number

 10-digit number starting with 9 (e.g. 9171234567).

Password

Confirm password

Must include uppercase, lowercase, number, and symbol.

Hospital record code

Enter the code from the slip you received at the PACD or your clinic (such as the Red Star Clinic). It shows you have a DRMC hospital record and links your records to this account. Your date of birth must match your hospital record. No code yet? Visit the PACD with a valid ID to get one.

Filled in from your QR code.

[DRMC staff with an invitation? Use it instead](#)

Credentials step with a hospital record code prefilled from the QR. Demonstration screenshot from the reference PDF.

The in-form **Scan QR** option depends on camera permission and browser BarcodeDetector support. A phone's normal camera link or manual typing remains usable when that button is unavailable.

Patients ordinarily need a hospital record code. Staff use invitations; the first Admin uses bootstrap. A government ID upload or an authenticator QR does not replace a hospital record code.

Patients do not need a staff role or mandatory staff MFA to access their own patient pages. They may enable an authenticator for their account.

14 Handle signup problems without duplicate accounts

Invalid, used, revoked or expired codes, already linked records and birth-date mismatch receive the same privacy-preserving verification message. Staff should inspect the code status and registry rather than expose another patient's information.

Situation	Correct next step
Birth date differs	Compare ID and record. Admin corrects an inaccurate record; otherwise patient re-enters the date.
Lost, expired or revoked code	Verify identity, then issue a new code for an unlinked record.
Redeemed code or linked record	Recover or sign in to the existing account; do not create a duplicate record to get another code.
Missing code	Patient needs a hospital record and desk-issued code. Do not turn off the requirement to bypass an error.
Wrong invitation email	Use the invited email; Admin reviews expired/revoked unused invitations.
No confirmation email	Check Development console, or real delivery/spam. Use the resend confirmation page.
Page failed after Create account	Check whether the account and record link already exist before repeating signup.

What is atomic and what can fail later

The Identity account is created first. A later **serializable database transaction** atomically redeems the code and sets `PatientRecords.PortalUserId`. This prevents two successful redemptions or two accounts claiming that record.

Account creation, uploaded-document storage, activity audit and email delivery are outside that linking transaction. The code explicitly compensates for failed redemption, invitation acceptance or document saving. Later audit or email failures can leave an account and linked record present.

If the account exists after an error, use **/Identity/Account/ResendEmailConfirmation** and account sign-in/password recovery. An Admin can inspect the registry link. Repeating Create account with the same code is not a general rollback or delivery-retry mechanism.

Existing accounts and supervised manual linking

If a patient already has a portal account, Admin can link an existing email-confirmed account through **Verified portal link**, after separately verifying identity. This is distinct from new-account code redemption.

The configuration **PatientRegistration:RequireHospitalRecordCode=false** permits signup without a code. Use it only for a deliberate supervised enrollment plan; such accounts do not gain clinical access until staff verify and link them. Keep the default true for normal operation.

Unlinking removes access to clinical records through that link. Deleting an account preserves the independent hospital record. Review administrative changes in staff audit history.

15 Maintain clinical records and public content

Adding patients and doctors does not populate visits, results or prescriptions. Enter approved records manually in the permitted staff sections, or import supported templates after reconciliation.

Task	Staff area and responsibility
Visit or care summary	Admin → Encounters, select the hospital record and approved physician information.
Laboratory result	Admin or Lab staff → Lab results; add parameters through Lab items.
Imaging study	Admin or Radiology staff → Radiology; enter approved study, status and release information.
Medication and dose times	Admin → Prescriptions, then Dose schedules.
Allergy	Admin → Allergies for the selected hospital record.
Public doctor	Admin → Doctors. Staff invitations and doctor listings are separate.
Public notice	Admin → Advisories; publish approved content and dates.
Audit review	Admin → Audit history. Review staff access and change entries.

Use the hospital patient picker, never an account display name as an ownership shortcut. Forms and imports save historical physician names; changing a directory doctor's name does not update past clinical entries.

What patients can see

PatientResults:ShowFullResults=false is the default. Full released laboratory values and imaging report sections remain gated even when availability is shown. Turning this on requires DRMC's disclosure decision.

For imaging, **Final** or **Amended** studies whose release time has passed are ready for claiming. Reports still in preparation show progress. Internal notes do not reach patient pages. The portal does not display imaging films; patients claim films/CDs through Radiology.

Clinical date/time inputs represent **Manila wall time** and are stored without an offset. Do not add UTC conversion to a legacy file unless its original timestamp meaning has been confirmed. Public advisory effectivity and audit/import/system times use UTC.

Content that still requires developer changes

Departments, OPD and Malasakit guides, privacy/terms, official links and facility guidance are currently code-defined content. Public advisories and doctor rows have Admin screens, but the whole site is not a content-management system.

Have the client verify chair names, rooms, telephone extensions, service hours, preparation instructions, release procedures and translated text. Demo content is not evidence of a live hospital integration.

16 Developer workflow and test data

Use a working branch and a pull request into master. Keep migrations in source control. Separate development, test, staging and live databases and storage folders.

Verify a code change

POWERSHELL

```
dotnet restore DrmcPatientPortal.slnx
dotnet build DrmcPatientPortal.slnx -c Release --no-restore -warnaserror
$project = 'src/DrmcPatientPortal'
dotnet ef migrations has-pending-model-changes --project $project
```

The last command checks model drift; it does not apply a schema change. If a developer intentionally changes the EF model, create and review a migration, then test the upgrade against a separate database. New installations apply committed migrations; they do not create a new Initial migration.

Run the SQL Server integration tests

Use a server where the test login may create and drop disposable databases. Omit a database name:

POWERSHELL

```
$env:DRMC_TEST_SQLSERVER = (
    'Server=localhost\SQLEXPRESS;Integrated Security=True;' +
    'Encrypt=True;TrustServerCertificate=True'
)
dotnet test DrmcPatientPortal.slnx -c Release
```

The factory creates uniquely named **DrmcPortalTests_...** databases and temporary storage, migrates and seeds them, then drops its test databases. Do not run this against a live hospital server.

At the reviewed commit, verification on Windows with SQL Server Express passed **111 tests**, with zero skipped or failed, and a Release build passed with zero warnings. These checks do not prove a client's server, mail provider, phone camera or backup recovery works; rehearse those in the client's environment.

Optional fixtures for a throwaway demo only

POWERSHELL

```
$env:DevelopmentFixtures__Enabled = 'true'
$env:DevelopmentFixtures__Primary__Email = Read-Host 'Demo patient email'
$env:DevelopmentFixtures__Primary__Password = Read-Host 'Demo password'
$env:DevelopmentFixtures__Empty__Email = Read-Host 'Empty demo email'
$env:DevelopmentFixtures__Empty__Password = Read-Host 'Empty demo password'
dotnet run --project src/DrmcPatientPortal --launch-profile http
```

Fixtures run only in Development, with explicit credentials and empty business, audit and import tables. They include example doctors and synthetic clinical records. Existing data makes the whole seed skip; it is not a repair or reseed tool.

Keep fixtures disabled for the client's approved roster and patient records. Remove every DevelopmentFixtures setting after a demo initialization. If you need another clean rehearsal, create another isolated development database rather than deleting mixed client data.

17 Prepare the client hosting environment

A hosted client installation uses its own SQL database, storage folders, secrets, DNS name and certificate. The concrete deployment path here is **Windows Server with IIS**. Local Docker is a database development option, not the production hosting stack.

Agree on installation details

Detail	Record the client's decision
Portal HTTPS address	Patient-reachable DNS name and certificate owner
SQL target	Server/instance, port if needed, database name and collation
SQL identities	Separate migration and runtime identities
App identity	Dedicated IIS application-pool identity and storage permissions
Files and keys	Persistent key ring, document, temporary and import staging paths
Notifications	SMTP provider and HTTPS SMS webhook credentials
Operational owners	Admins, patient desk, backup/recovery owner and deployment operator
Approved content	Doctor roster, titles, department details and patient disclosure policy

Production defaults to preserve

Use **ASPNETCORE_ENVIRONMENT=Production**, **Identity:RequireConfirmedAccount=true**, **PatientRegistration:RequireHospitalRecordCode=true**, and **PatientResults:ShowFullResults=false** until disclosure is approved. Keep Development fixtures off.

Serve through HTTPS with a trusted certificate. Outside Development the app uses secure cookies and HSTS. A plain HTTP deployment cannot support the intended authenticated flow.

Notification settings

Environment key	Supply
Notifications__Email__Host	SMTP hostname
Notifications__Email__Port	Provider's SMTP port; default 587
Notifications__Email__UseSsl	Provider's supported TLS setting; default true
Notifications__Email__Username	SMTP credential if required
Notifications__Email__Password	SMTP secret
Notifications__Email__FromAddress	Approved sender address
Notifications__Sms__Endpoint	HTTPS webhook URL
Notifications__Sms__ApiKey	Webhook bearer-token secret
Notifications__Sms__SenderId	Approved sender ID; default DRMC

Both email and SMS settings are validated outside Development, including Staging. Email startup validation requires Host, positive Port and FromAddress; SMS validation requires an HTTPS endpoint and API key. Startup validation does not prove delivery, so send real test messages before handover.

The implemented SMS integration POSTs JSON **to**, **message** and **senderId**, with **Authorization: Bearer <ApiKey>**. It is a generic webhook contract; an arbitrary provider may need an adapter. SMS notification configuration does not enable SMS OTP sign-in.

17 Prepare the production database and schema

The DBA creates a separate empty portal database with **Latin1_General_100_BIN2**. The examples below use **DrmcPatientPortal**; choose the actual client name and verify every target.

SQL

```
CREATE DATABASE [DrmcPatientPortal]
COLLATE Latin1_General_100_BIN2;
```

Use a deployment/migration identity with the schema permissions needed for upgrades. Use a separate runtime identity without server administration or schema-change permissions.

Generate a migration script for DBA review

On the build machine, using a working Development design-time configuration:

POWERSHELL

```
$project = 'src/DrmcPatientPortal'
New-Item -ItemType Directory -Path artifacts -Force | Out-Null
dotnet ef migrations script --idempotent --project $project -o artifacts/drmc-schema.sql
```

Review the script against the deployed commit. It does not create the target database; the DBA creates it first. For an upgrade, back up the existing database, stop older builds and apply only the reviewed release's migration script.

The DBA can run the SQL file in SSMS with the correct database selected, or with sqlcmd. Example using Windows authentication:

POWERSHELL

```
sqlcmd -S <sql-server> -d DrmcPatientPortal -E -I -b -i artifacts/drmc-schema.sql
```

Replace the server placeholder. **-I** enables quoted identifiers, required for the schema's filtered indexes; **-b** reports SQL errors through a failing exit status. Do not treat a failed or partly applied script as a successful upgrade.

Map a runtime identity

If the DBA has already created the SQL login `drmc_app`, a database user and baseline read/write membership can be set as follows:

SQL

```
USE [DrmcPatientPortal];
CREATE USER [drmc_app] FOR LOGIN [drmc_app];
ALTER ROLE [db_datareader] ADD MEMBER [drmc_app];
ALTER ROLE [db_datawriter] ADD MEMBER [drmc_app];
```

This is broad database read/write access, not a fine-grained permission design. The DBA reviews the runtime permissions and tests startup, Identity roles, CRUD, audit and imports. Configure the app's runtime connection, not the migration identity:

TEXT

```
Server=<host>,1433;Database=DrmcPatientPortal;User Id=drmc_app;
Password=<runtime-secret>;Encrypt=True;TrustServerCertificate=False
```

The displayed connection wraps at a semicolon; supply it as one value. Use a trusted SQL certificate and correct hostname. With Windows authentication, SQL sees the hosting process identity, not the developer who deployed the files; map the actual server identity deliberately.

17 Publish and install on IIS

This procedure assumes a Windows server chosen by the client's IT team and an x64 application pool. Use a supported Windows/.NET combination.

Publish from the reviewed source

POWERSHELL

```
dotnet publish src/DrmcPatientPortal -c Release -r win-x64 --self-contained false -o artifacts/publish
```

This produces a framework-dependent release. Copy the entire publish output, including generated **web.config**, configuration files, browser assets and required native files. Do not copy only the DLL.

Prepare IIS

1. Install the Windows Server **Web Server (IIS)** role.
2. Install the **.NET 10 Hosting Bundle** after IIS. If it was installed first, repair it after enabling IIS.
3. Create a dedicated application pool and choose **No Managed Code**. Keep the application and pool architecture x64.
4. Create a site whose physical path points to the publish folder, for example **C:\Sites\DRMCPortal**.
5. Set an HTTPS binding for the intended portal hostname with a valid certificate; configure DNS and network access.
6. Grant the pool identity read/execute on the deployed application and Modify on the separate persistent storage directories.
7. Set host configuration and secrets before starting the app. Apply the database schema first.
8. Start the site, open the public HTTPS URL, then complete production Admin bootstrap through that URL.

The deployment mechanics follow Microsoft's [IIS publishing tutorial](#) and [IIS hosting guidance](#).

Avoid a deployment-dependent data path

Keep files and keys under **C:\DRMCData**, or another controlled persistent root, rather than under the versioned publish folder. The app creates storage directories where permitted, but the hosting identity must be able to write there.

If using DPAPI current-user key protection, enable **Load User Profile** and preserve the app-pool identity/profile. Chapter 18 explains why copying these keys to another machine is not a complete recovery strategy.

A hosting server does not inherit environment variables from the PowerShell window on your development PC. Site settings must reach the actual IIS worker process. See the next page.

Acceptance condition: The app starts in Production, reaches the intended fully migrated database, writes to the intended persistent folders, and delivers confirmation/invitation email through the client's provider.

17 Configure the hosted process and QR address

Use the deployment system or site-scoped IIS environment settings to inject configuration into the worker process. Restrict access to any server-local files containing secrets; do not commit those files to Git.

For example, the generated web.config's existing aspNetCore element can contain these **non-secret** environment values. Merge the environmentVariables child into the generated configuration rather than replacing the whole file:

XML

```
<environmentVariables>
  <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
  <environmentVariable name="AllowedHosts" value="portal.your-domain.example" />
  <environmentVariable name="DataProtection__KeyRingPath"
    value="C:\DRMCDData\Keys" />
  <environmentVariable name="PatientDocuments__RootPath"
    value="C:\DRMCDData\PatientDocuments" />
  <environmentVariable name="PatientDocuments__TemporaryPath"
    value="C:\DRMCDData\TempUploads" />
  <environmentVariable name="Imports__StagingPath"
    value="C:\DRMCDData\ImportStaging" />
</environmentVariables>
```

Replace the example host. Supply the connection and notification secrets through the approved host mechanism. Recycle the application pool after configuration changes. IIS environment injection is described in [Microsoft's ASP.NET Core Module guidance](#).

Bootstrap on the server

Temporarily add **AdminBootstrap__Enabled=true** and the first Admin's email to the actual hosted process settings. Follow chapter 8 using real email delivery. Restart only after email confirmation and authenticator setup, then remove both bootstrap values, restart again and sign in with MFA.

Keep generated links usable by patients

Patient code and staff invitation URLs use the incoming request's scheme and host. Staff should open Admin through the same patient-reachable **HTTPS hostname** used for the portal.

IIS integration handles its own forwarding. Additional reverse proxies require reviewed forwarding configuration: this repository does not explicitly configure general forwarded-header middleware in Program.cs. Passing headers alone is insufficient if the app does not consume trusted proxy headers. Validate redirects, cookie security and both generated QR/link types before deployment.

A root-hosted site is the simplest path here. Subpath or additional proxy hosting requires its own route and link tests.

Keep the import worker running

The worker runs inside the web application; it is not a separate Windows service. Configure IIS initialization and idle behavior if jobs must run without browser traffic, and rehearse an idle/restart import. Do not promise continuous background execution while the app pool is stopped.

Two same-build processes may overlap during recycle only with the same database, identity, protection settings, key ring and staging directory. General horizontal scaling and mixed-version workers are not supported.

18 Back up and recover the installation

The SQL database is only part of the installation. Encrypted document files and their historical Data Protection keys must be recoverable together.

Back up these components

Component	Why it matters
SQL database	Accounts, role memberships, registry, clinical data, token hashes and audit/import metadata
Data Protection key ring	Decrypts document files and unfinished import staging; also protects cookies and tokens
Patient document storage	Encrypted ID images represented by database metadata
Temporary and import staging	Needed according to the chosen handling of unfinished registrations and imports
Deployed source/build version	Identifies compatible code and migrations for recovery
Protected host configuration	Database, mail/SMS access and stable storage/identity settings

Have the DBA perform and verify SQL backups. Backup file paths refer to the SQL Server machine, not necessarily the SSMS workstation. Agree on retention, access and a protected off-server copy. Backing up only a Docker volume or copying running database files is not a demonstrated recovery procedure.

Choose key protection deliberately

The default **DataProtection:ProtectKeysWithDpapi=false** leaves key secrets unprotected at rest. Folder permissions and protected backups are essential. Enabling **true** on Windows converts existing plaintext secrets and protects new keys with current-user DPAPI.

DPAPI ties decryption to the Windows account and machine. Preserve the original identity/profile and keep the setting enabled afterward. Turning it off does not decrypt the key ring; startup rejects protected keys with the flag off.

Copying DPAPI-protected XML to another machine or user does not by itself produce usable recovery. A portable certificate-based or DPAPI-NG strategy is not implemented in this version. Read the repository [key protection and recovery section](#) before choosing a strategy.

Rehearse restoration before handover

Restore to a controlled environment under the chosen compatible identity. Confirm the expected migration state, a sample authorized encrypted document, sign-in, correct patient ownership and import behavior. Use synthetic or explicitly approved test data.

Lost historical keys can make old documents and unfinished imports unreadable. Do not delete keys to solve a startup error or reset a failed import state to bypass decryption.

Upgrades and failed imports

Back up, stop/drain older builds, apply reviewed migrations, deploy the matching build, restart and run acceptance checks. A schema rollback or older build needs separate compatibility review.

Queued import phases survive restart. Running phases with expired leases fail without automatic replay. Cancel and upload reviewed replacements; running phases cannot be cancelled. Unfinished batches expire seven days after upload. Keep database and storage together rather than manually editing job states.

19 Troubleshoot setup and staff access

Symptom	Check and action
dotnet not found or target framework error	Install the .NET 10 SDK, reopen the terminal and check dotnet --version.
dotnet ef not found or version warning	Install/update dotnet-ef 10.0.11 and check PATH/tool version.
CONFIGURE_VIA_ENVIRONMENT in a connection error	The placeholder was used. Set the connection in the actual process environment.
SQL login or network error	Check service/instance, server/port, auth mode and SQL identity. An IIS integrated connection uses the app identity.
Initial migration rejects collation	Use a separate empty database with Latin1_General_100_BIN2. Do not discard a populated database to fix this.
Missing table or column	Verify all eight migrations against the same database used by the app. Startup does not apply them.
Production notification validation error	Supply SMTP host/port/from address and HTTPS SMS endpoint/API key to the hosted worker.
Bootstrap refuses startup	Configured account exists but email/2FA is unfinished or it is locked out. Disable bootstrap, complete setup, then promote.
Admin returns access denied	Check current role, confirmed email, 2FA, lockout and MFA sign-in. Forget remembered browser, sign out/in with authenticator.
Invited staff sees no Administration link	Finish email confirmation and 2FA, then sign out/in. Check invitation was not revoked.
Email confirmation returns HTTP 400	Copy a fresh link. Development console HTML requires replacing & with &.
Authenticator code fails	Check device time and correct account entry; wait for a current code. Repeated failures may trigger lockout.
No doctors in public listing	Confirm successful load, IsActive=true, and selected department/filter.
Doctor import fails	Check required Title, canonical Department, ordered headers, UTF-8 and duplicates.
OCR unavailable	Continue with manual ID entry; validate native libraries separately.
Storage access or decryption error	Verify actual process identity, persistent paths, permissions and compatible historical keys.
QR points to localhost or HTTP	Generate through the real HTTPS host and verify proxy scheme/host handling.
Import remains queued	Ensure the app process and worker are running; inspect state, eligibility, storage and logs.

Use chapter 14 for patient enrollment and existing-account problems. An error after signup may leave the account present; resend confirmation instead of repeatedly claiming the same code.

Record the timestamp, deployment commit, affected route and sanitized error for the developer. Keep passwords, code slips, real patient fields and secrets out of shared screenshots and issue reports.

20 Rehearse the complete workflow and hand over

Perform this on a separate staging or local rehearsal database with synthetic people. Repeat the hosting-specific checks on the actual client environment before opening enrollment.

Acceptance checklist

1. **Code and database:** Record the source commit, SQL server/database and all eight migration IDs. Build succeeds and public pages load.
2. **Storage:** Confirm document, temporary, key and import paths are persistent and writable by the actual app identity.
3. **First Admin:** Create the intended account, confirm email, enable authenticator, promote on restart, remove bootstrap and verify MFA access.
4. **Staff:** Invite one Lab, Radiology and Patient services account. Confirm email and 2FA, verify each permitted area, and verify unrelated areas deny access.
5. **Roster:** Load a small approved or synthetic doctor file, compare counts and check canonical department filters. Confirm blank Title and duplicate imports are rejected.
6. **Registry:** Create two unrelated test records with different birth dates and sample clinical entries.
7. **Patient linking:** Issue a code for the first record, register with its matching birth date, confirm email and sign in. Confirm only its data appears.
8. **Negative enrollment:** Use separate codes to try wrong birth date, revoked/expired codes and replay after success. Check each rejects enrollment as expected.
9. **QR reachability:** Scan a new slip from a phone. Confirm patient and staff links use the correct HTTPS host and reach the intended app.
10. **Notifications:** Confirm real email delivery and the configured SMS webhook separately. Check resend confirmation and invitation fallback.
11. **Results policy:** With default full-results setting false, confirm availability behavior and concealed detailed result sections.
12. **Operations:** Test a supported import, app recycle/idle behavior, a backup restore and authorized encrypted-document access.

Complete the handover record

Item	Client owner or value to record
Source repository and deployed commit	Client code owner and release
Database and migration operator	Server, database and DBA
Portal hostname/certificate	DNS and renewal owner
Admin and staff access	Named owners and offboarding process
Directory approval	Roster, Title values and canonical departments
Patient registration desk	Issuing points and identity-check procedure
Recovery	Backup schedule, key strategy and tested MFA recovery owner
Content and disclosure	Approved guides, translations and full-results decision
Known remaining work	Clinical integrations, content changes and provider adapters

Keep credentials in the approved secret system, not this handover table. An independent installation is ready when these checks and responsibilities are recorded, not merely when the home page opens.

Appendix A Migration and source reference

This edition has **eight** SQL Server migrations:

Migration ID	Purpose
20260929123803_InitialSqlServer	SQL Server baseline
20261004153506_PatientRegistryOwnership	Independent hospital records and clinical ownership
20261004162211_AdminConcurrencyAudit	Concurrency and staff audit
20261005042232_AdminImportBatches	Import metadata
20261005070928_AdminImportLeases	Durable worker leases
20261006024312_AddRadiologyStudies	Dedicated imaging studies
20261006164919_AddPatientRegistrationCodes	Patient code enrollment
20261007035528_AddStaffInvitations	Staff invitation enrollment

Check in the intended database, using SSMS:

SQL

```
SELECT DB_NAME() AS DatabaseName,
       DATABASEPROPERTYEX(DB_NAME(), 'Collation') AS CollationName;
SELECT MigrationId
FROM dbo.__EFMigrationsHistory
ORDER BY MigrationId;
```

Or list migrations through EF with the configured connection:

POWERSHELL

```
dotnet ef migrations list --project src/DrmcPatientPortal
```

Main source locations

Source	What it defines
Program.cs and appsettings files	SQL provider, Identity, notifications, storage and policies
Data/ApplicationDbContext.cs	Tables, ownership, indexes, collation and relationships
Data/Migrations	Committed schema history
Areas/Admin/Security/AdminSecurity.cs	Live staff eligibility and startup bootstrap
Services/StaffInvitations.cs	Email-bound acceptance and role activation
Services/HospitalRecordCodes.cs	Patient code format, hashing and lifetime
Areas/Identity/Pages/Account/Registrar.cshtml.cs	Account creation and record redemption
Models/ClinicalDepartment.cs	Static department catalog
Areas/Admin/Services/ImportTemplates.cs	Current template headers and validation version

Paths in this table are relative to **src/DrmcPatientPortal**. The design documents explain rationale; current code and tests decide what is implemented.

Appendix B Useful addresses

Add these paths to your own portal origin. Local HTTP is **http://localhost:5095**; local HTTPS is **https://localhost:7104**. Production uses the client's HTTPS hostname.

Path	Purpose and access
/Directory	Public doctor listing
/Directory/Doctor/{id}	Public individual profile
/Directory/Department?name=<department>	Public department page; URL-encode the name
/Identity/Account/Register	Patient code, staff invitation or first Admin signup
/Identity/Account/Login	Sign in
/Identity/Account/ResendEmailConfirmation	Request a new confirmation email
/Identity/Account/ForgotPassword	Account password recovery
/Identity/Account/Manage/TwoFactorAuthentication	Signed-in authenticator management
/Patient/Home	Signed-in patient dashboard
/Patient/Radiology	Patient's own imaging availability
/Admin	Eligible staff dashboard
/Admin/Patients	Admin registry and verified portal links
/Admin/RegistrationCodes	Admin/Patient services issue and revoke codes
/Admin/StaffInvitations	Admin staff invitations
/Admin/StaffAccess	Admin staff role grants/revocations
/Admin/Doctors	Admin directory maintenance
/Admin/Imports	Admin templates, reconciliation and batch status
/Admin/LabResults	Admin/Lab staff laboratory records
/Admin/LabResultItems	Admin/Lab staff laboratory parameters
/Admin/RadiologyStudies	Admin/Radiology staff imaging entries
/Admin/ClinicalEncounters	Admin visits
/Admin/Prescriptions	Admin medication records
/Admin/MedicationDoseSchedules	Admin dose times
/Admin/PatientAllergies	Admin allergies
/Admin/Audit	Admin staff audit history

Use sidebar links and patient/parent pickers rather than guessing an ID or record route. Verify these paths again when upgrading to a build that changes routing.

Normal staff access requires current confirmed email, 2FA, nonlocked status, permitted role and a sign-in carrying MFA proof. Granting a role does not eliminate those checks.

Patient registration QR fragments use **#code=...**. Staff invitation fragments use **#invite=...**. Authenticator enrollment uses a separate QR for the authenticator secret. These three QR types have different purposes.

Appendix C Review of the supplied reference PDF

Assessment: The attached 28-page DRMC_Patient_Portal_Setup_Guide.pdf is substantially complete for the reviewed commit. It already explains cloning, SQL Server, bootstrap, staff invitations, doctors, imports and patient codes. The core onboarding approach matches the code.

This edition retains those workflows and corrects details that could mislead a new team:

Reference location	Correction or clarification
Pages 5 and 9	Use SSMS or VS Code with MSSQL for a new installation; Azure Data Studio retired in February 2026.
Page 7	Apple silicon amd64 emulation is not a Microsoft-supported SQL container environment.
Page 12	Custom authenticator setup does not generate/display recovery codes. Use the built-in generation and recovery pages described in chapter 8.
Page 15	Invitation expiry limits unused acceptance; an accepted Awaiting setup invitation can activate later unless revoked.
Page 19	Identity account creation is separate from the serializable transaction that redeems the code and links the record.
Page 22	Compensation is narrower than every later failure. Audit/email failure can leave a created account and linked record; check state and resend confirmation.
Page 24	IIS has built-in forwarding integration; additional proxies need explicit reviewed handling. Passing headers alone is insufficient.
Page 26	Inactive doctors disappear from listings, but the direct profile route currently still loads by ID.
Command and CSV blocks	Long commands should have intentional continuations; exact CSV headers remain one line in the supplied companion file.

Practical additions

This edition gives explicit separate-database naming, a local target check, client roster Title requirements, shared UserSecretsId behavior, precise import reconciliation/approval labels, the unsupported dedicated radiology import, and a concrete IIS publication and configuration walkthrough.

It also separates fresh installation from upgrading or restoring existing data and adds a handover and recovery rehearsal. Clearing a throwaway environment is not a general remedy for a populated database.

Repository documentation differences

At this commit, **ImportTemplates.ValidationVersion is 1.1.0**, while docs/import-templates.md says 1.0.0. The worksheet/template names still end in **_v1**. Download templates from the running application and review current validation messages.

The README prescribes a fully migrated database. Startup code does not implement an explicit check of every pending migration in every environment. This guide treats all migrations as an installation prerequisite and verifies history rather than promising a specific pending-migration rejection.

No authoritative client doctor roster was supplied for this review. Names and departments alone need approved Title values; real doctors, schedules and credentials have not been populated as part of creating this guide.

Appendix D Sources and verification

Application evidence

The working checkout's commit matched live GitHub **HEAD on master**:

d86ddb61f05fac0f4bf007932407998d3f790cdb

- Repository at the reviewed commit
- README
- Import contract
- Hospital record and staff onboarding design
- Current startup, Identity, Admin, template and ownership source files, and the existing integration tests.

The supplied reference PDF was read across all **28 pages** and selected page images were inspected. Four demonstration screenshot regions are reused unchanged for orientation. They are not captures of the client's deployment or evidence of its actual accounts or patient data.

Checks performed for this edition

Check	Observed result
SDK	.NET SDK 10.0.400; application/runtime dependencies 10.0.11
Package restore	Successful
Release build with warnings as errors	Successful, zero warnings and errors
EF pending-model-change check	No model changes since the last migration
SQL Server integration suite	111 passed, 0 failed, 0 skipped
Separate recovery-page rehearsal	Generated ten codes, used one for password-plus-recovery Admin access, confirmed nine remained
Test database isolation	Existing test factory used uniquely named disposable SQL databases
Roster loading	Documented from source; no client data was inserted

The integration suite exercises first-Admin signup eligibility, staff invitations, role activation/access, patient code validity/replay/concurrency, record ownership, email verification and clinical workflows. Client hosting, provider delivery, ARM execution, camera support and recovery must be verified in that client's environment.

External setup references

- SQL Server installation wizard
- SSMS installation
- Azure Data Studio retirement
- SQL Server editions
- SQL Server container support
- Publish ASP.NET Core to IIS
- IIS hosting
- ASP.NET Core Module environment configuration

Companion file: **docs/setup-guide-samples/doctors-demo.csv**. Editable guide source: **docs/setup-guide.md**. Final client deliverable: **docs/DRMC_Patient_Portal_Full_Setup_Guide.pdf**.